

WEEK 1 • DAY 2

Enterprise Security Baseline

RBAC, ABAC, Data Markings, Object-Level Security & Auditability

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture — no live Palantir platform work today
Session Format	One combined live session for India & Europe — 2:00–3:00 PM IST / 10:30–11:30 AM CEST
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3+)	Build a 7-object Insurance Ontology — Policyholder, Insurance Policy, Claim, Claim Document, Payment, Adjuster, Investigation — with RBAC/ABAC applied

Day 1 Recap & Coverage Check

What we covered, and what today bridges before we start Security



Covered Day 1

Data Products, Ontology-first design, Domain-Driven modeling, Object/Link Type anatomy, branching, incremental processing



Bridged Today, Before Security

Functions and Action Types — covered briefly Day 1, deepened in the next few slides because today's security topics depend on them



Tracked for Day 3–5

Foundry/AIP/Apollo platform architecture, Pipeline Builder internals, and remaining Ontology-engineering depth — folded into the Day 3 lab guide and Day 4–5 discussion, not dropped

Today's Roadmap

One short bridge, then nine security topics

→	Bridge: Functions & Action Types	The two concepts every permission topic today assumes you already have
1	RBAC	Role-Based Access Control — who can do what, per Project
2	ABAC	Attribute-Based Access Control — eligibility-based, independent of Role
3	Data Markings	How ABAC is actually applied to sensitive resources
4	Object & Property-Level Security	Fine-grained control within the Ontology itself
5	Function & Action Permissions	Governing who can execute logic and trigger change
6	Purpose-Based Access Patterns	Access tied to why data is being used, not just who's asking
7	Security Propagation	How access rules travel across pipelines and the Ontology
8	AI & Agent Permission Boundaries	Constraining what a Copilot or Agent can reach
9	Auditability	Proving, after the fact, who did what

By the End of Today

Five governance layers that must exist before any AI capability touches this Ontology



Distinguish RBAC from ABAC

Discretionary access control vs. mandatory, eligibility-based access



Apply object-, property-, and purpose-based security

Protect sensitive fields and constrain access by intended use, not just identity



Set boundaries on what an Agent or Copilot can reach

Before Week 2 builds a Copilot on top of this Ontology



Explain how security propagates and gets audited

Across pipelines, the Ontology, and every governed action

BRIDGE

Functions & Action Types

The foundation every permission topic today assumes

Why This Bridge Comes First

“Function and action permissions” only makes sense once you know what a Function and an Action Type actually are. Day 1 introduced Action Types briefly as part of Object Type anatomy — today we go one level deeper before applying security to them.

Functions & Action Types, Side by Side

The two concepts every permission topic today assumes

Function — Computes, doesn't change

Takes Ontology data in, returns a computed value (an eligible claim range, a risk score) without writing anything back

Action Type — Defines a governed change

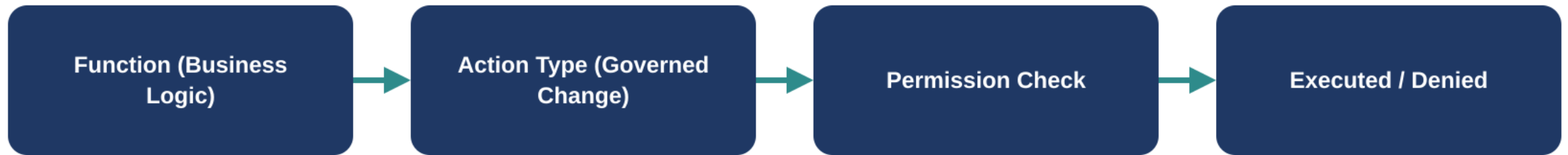
Parameters, submission criteria, and side effects — e.g. ApproveClaim: criteria (amount $\leq$ policy limit), side effect (updates Claim.status)

The Security Connection

Who can invoke a Function, and who can trigger an Action Type, are both security decisions — covered later today

From Business Logic to Governed Change

The chain security sits on top of



Everything from here on is about that last box — how “Permission Check” actually gets decided.

TOPIC 1

Role-Based Access Control (RBAC)

Who can do what, per Project

What Is RBAC?

Discretionary access, granted per Project

RBAC grants permissions based on a user's assigned Role within a Project — the primary security boundary in the platform. It answers: what can this person do here?

Roles in Practice

The default Roles, applied to this week's Insurance Ontology

Owner	Full control — manage the Project, its content, and who has access to it
Editor	Create and modify content — e.g. an Adjuster updating an Investigation record
Viewer	View content, no edit rights — e.g. an auditor reviewing Claim history
Discoverer	See that the Project exists and request access — cannot view content

TOPIC 2

Attribute-Based Access Control (ABAC)

Eligibility-based access, independent of Role

What Is ABAC? — Data Markings in the Platform

Mandatory, eligibility-based access, independent of Role

A user must satisfy every attribute-based eligibility rule (a Marking) applied to a resource to access it at all — an Owner Role does not override a missing attribute match.



A Marking is a binary eligibility rule

A user either satisfies it or doesn't — there's no partial access, regardless of Role



Centrally managed, consistently applied

The right tool for data needing the same protection everywhere it appears — like PII

RBAC vs. ABAC — When Each Applies

They're not alternatives — they're layers

RBAC (Roles)

Best for: what a person can generally do within a Project. Coarse-grained, project-scoped.

ABAC (Markings)

Best for: categories of data needing the same protection everywhere — PII, high-value claims. Fine-grained, resource-scoped.

Worked Example: Marking Sensitive Claims

Applying both layers to this week's Ontology



High-value claims get a Marking

Claims above a threshold amount require a “High-Value Claim” Marking to view, regardless of Role



Adjusters get Editor via RBAC

An Adjuster's Role lets them edit Claim and Investigation objects in general



Both must pass

An Adjuster without the High-Value Marking still cannot open a high-value Claim, even with Editor Role

TOPIC 3

Object & Property-Level Security

Fine-grained control within the Ontology itself

Object-Level Security

Controlling whether a user can see an object at all

What It Controls	Whether a specific Claim, Investigation, or Payment object is visible to a given user
Dynamic Security	Computed at query time — e.g. a Claim is visible only to its assigned Adjuster
This Week's Objects	Applies across all seven — Policyholder, Policy, Claim, Claim Document, Payment, Adjuster, Investigation

Property-Level Security

Protecting a field, even when the object itself is visible



Object visible, property hidden

A user might see a Claim exists, but not its fraud-risk score specifically



Security markings display, not hide silently

Users can see that a property is protected without seeing its value — restrictions are transparent, not silent



Common target: Payment and Claim Document

Financial amounts and uploaded documents are typical candidates for property-level protection

TOPIC 4

Function & Action Permissions

Governing who can execute logic and trigger change

Function & Action Permissions

Controlling who can compute, and who can trigger change

Function Permissions	A Function is not automatically open — even read-only computation can expose data. Scoped by Role and Marking, same as object viewing
Action Permissions	An explicit list of Roles or Markings eligible to submit a given Action Type; submission criteria combine with permission — both must pass
Example: ApproveClaim	Only Adjuster or Senior Adjuster Roles, and only for claims within the assigned Adjuster's own caseload

TOPIC 5

Purpose-Based Access Patterns

Access tied to why data is being used, not just who's asking

Purpose-Based Access Patterns

The same person, different access, depending on the task



Identity alone isn't always enough

The same Adjuster may be allowed to view a Claim Document for investigation, but not for an unrelated audit request



Purpose becomes part of the access decision

The declared reason for access — investigation, audit, servicing — is evaluated alongside Role and Marking



Why this matters for AI

An Agent's purpose (e.g. “fraud investigation”) should scope what it's allowed to retrieve, not just its Role

TOPIC 6

Security Propagation

How access rules travel across pipelines and the Ontology

Security Propagation — Pipelines & Ontology

Protection doesn't stop at the source dataset



Markings flow downstream through pipelines

A Marking applied to source claim data should propagate through every transform — the common failure is a join silently dropping it



Link Types don't bypass security in the Ontology

Navigating Policyholder → Claim via a Link Type doesn't grant access to a Claim the user couldn't see directly



Consistency is the goal, in both directions

The same object should be equally protected whether reached directly, through a relationship, or via a derived dataset

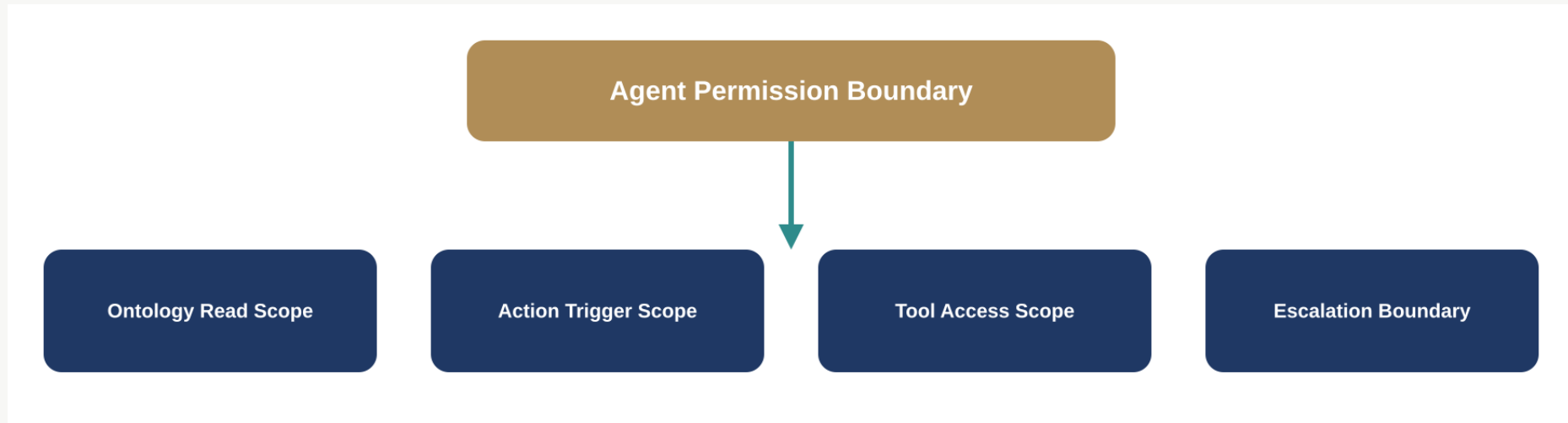
TOPIC 7

AI & Agent Permission Boundaries

Constraining what a Copilot or Agent can reach

AI & Agent Permission Boundaries

An Agent's reach should be defined, not assumed



Every AI capability from Week 2 onward inherits these boundaries — an ungoverned Ontology produces an ungoverned Copilot.

Worked Example: Constraining a Copilot's Reach

Applying today's layers to next week's build

Ontology Read Scope	The Week 2 Claims Copilot can read Claim, Policy, and Policyholder — not Investigation, which stays SIU-only
Action Trigger Scope	The Copilot can summarize and flag — it cannot submit ApproveClaim; that stays a human-triggered Action
Escalation Boundary	A low-confidence response routes to a human, rather than the Copilot guessing

TOPIC 8

Auditability

Proving, after the fact, who did what

Auditability of Data & Action Access

The record that makes every other layer verifiable

Every audit log answers four questions: who performed an action, what the action was, when it happened, and where it occurred.



Append-only by design

Audit records cannot be altered after the fact — the record is the record



Covers data access and Action submission

Both “who viewed this Claim” and “who submitted ApproveClaim” are captured

Audit Logs in Practice

What you'll actually check when something goes wrong

“Who saw this claim?”

Trace every access event against a specific Claim or Claim Document object

“Who approved this payout?”

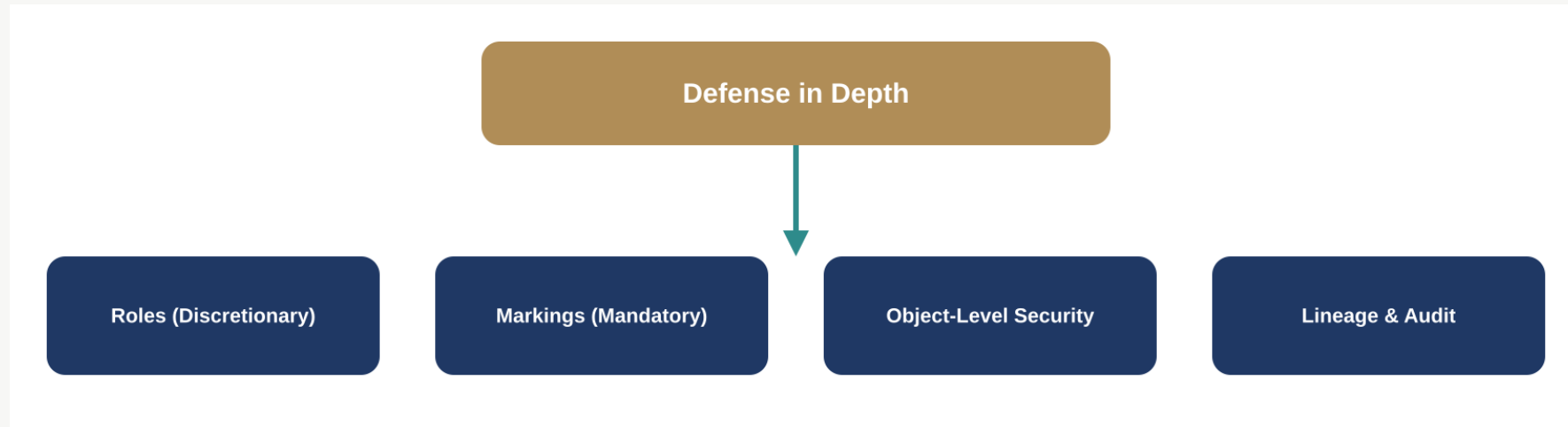
Trace the ApproveClaim Action submission, including the actor and the state before/after

“Did the Agent stay in scope?”

Trace an Agent's read and Action-trigger events against its declared permission boundary

Putting It Together — Defense in Depth

No single layer today is sufficient on its own



A strict Role doesn't compensate for a missing Marking. A Marking doesn't compensate for weak object-level security. Enterprise deployments layer all of them, then rely on audit to verify the layering is actually working.

Recap & What's Next

What We Covered Today

- Functions & Action Types (bridge)
- RBAC and ABAC / Data Markings
- Object-, property- & purpose-based security
- Function & Action permissions
- Security propagation, pipelines to Ontology
- AI & Agent permission boundaries, auditability

Day 3 — Self-Paced Lab

- Build the 7-object Insurance Ontology
- Configure incremental claims-transformation pipeline
- Apply RBAC and ABAC controls from today
- Test restricted access to high-value claims
- Review lineage and downstream dependencies



Questions

Rajesh Pasham